

Chapter: 8 User Authentication

I. What is User Authentication

The basic building block and the primary security line is user authentication. The most forms of access control and user transparency are focused on user authentication. As shown in a following page, user authentication is specified by RFC 4949 (Internet Security Glossary).

User Alice Toklas, for example, may have the ABTOKLAS user identifier. This information must be saved on every Alice server or computer System that system managers and other users can use and know about. The password, which is kept secret (only Alice and the system) for a standard item of authentication information associated with this user ID is. Where nobody can acquire or conjecture the Alice password, Alice User ID and Password combination allows administrators to mission-specific access to Alice and to audit their operation. As Alice's ID is not secret, system users can submit their email, but nobody can pretend to be Alice because their password is secret.

Identification is basically the means by which a user gives the system the claimed identity; user authentication is the means by which the validity of the claim is defined. Please notice that user authentication is not authenticated for messages. Chapter 12 describes message authentication as a mechanism for checking that the contents of a message received have not been changed and that a source is authentic. This chapter is about user authentication alone.

There are four ways to authenticate the identity of a user that can be used alone or in combination:

- **Something the individual knows:** Examples include a password, a personal identification number (PIN), or answers to a prearranged set of questions.
- **Something the individual possesses:** Examples include cryptographic keys, electronic keycards, smart cards, and physical keys. This type of authenticator is referred to as a *token*.
- **Something the individual is (static biometrics):** Examples include recognition by fingerprint, retina, and face.
- **Something the individual does (dynamic biometrics):** Examples include recog- nition by voice pattern, handwriting characteristics, and typing rhythm.

All these approaches will ensure safe user authentication, correctly implemented and used. However, there are difficulties in each process. An opponent can concoct a password or steal it. An opponent may likewise build or steal a token. Similarly. A user can lose a token or forget a password. In addition,



there is a considerable administrative overhead to maintain and secure device password and token information. There are a range of issues with biometric authenticators, including fabricating wrong positive and wrong negatives, welcoming consumers, costing and convenience. The most important methods for network-based user authentication include encryption keys and something that the person knows, like a password.

Mutual Authentication

Mutual authentication protocols are an important field of operation. Such protocols allow communicators to mutually satisfy each other with the identity of each other and share session keys. In previous chapters, this subject was investigated. The emphasis was on the main distribution. Here again, we look at the broader consequences of authentication.

There are two problems that are fundamental to the problem of authenticated key exchange. The important identity and session key information must be shared in encrypted form in order to prevent masquerading and avoid compromising of session keys. This requires that secrets or public keys are previously accessible and can be used for this reason. The second question, timeliness, is relevant due to the threat of monthly playback. These replays might, at worst, allow an opponent to compromise a session key or to personify another party successfully. A good replay can at least interrupt operations by sending messages to parties, but not real.

The following examples of **replay attacks**:

1. The easiest replay attack is one in which the adversary can easily copy and later replay a post.

2. An opponent will play a timed message in the current time window. This incident can be logged if both the initial and the replay arrive within the time frame.

3. As with example (2), an opponent can play back a timeline message in the valid time frame, but also delete the original message from the opponent. The repeat can not therefore be observed.

4. Another assault requires an unmodified reverse replay. This is a return to the sender of the post. This attack is possible if symmetric encryption is used to identify the difference between messages received on content and messages sent.

A sequence number must be added to each message used in an authentication exchange to deal with replay attacks. Only if the sequence number is in correct order will a new message be accepted. The problem with this method is that every party is needed to track each claimant's last sequence number. Due to the overhead sequence numbers are not commonly used to authenticate and exchange key numbers. Rather, two common approaches are used as one of the following:

Time stamps: Party A can only approve a new message if the message includes a time mark which is sufficiently close to A's current knowledge in A 's judgement. This solution involves synchronising the clocks between the different participants.



To maintain synchronization between different processor clocks, some type of protocol is required first. In order to cope with network faults, this protocol must both be fault tolerant and safe to cope with harsh attacks. Second, if a temporary failure of synchronisation arises from a clock fault in one of the groups, the potential for a successful attack will occur. Finally, the distributors are not required to maintain precise synchronizations due to the complex and unpredictable nature of the network delays. Thus, any pro-cedure time stamp must provide enough time for a window to handle network delays but small enough to reduce attack opportunities.

Challenge / Response: Party A sends B a nonce (challenge) and requires that the message (response) received by B contain the correct nonce value, awaiting the fresh message from B.

The challenge response method, on the other hand, is not suited to an excessive form of application as it involves an overhead handshake for any unconnected transmission that essentially denies the primary existence of a transaction without connection. For such applications, the best approach may be to rely on a safe time server and to try consistently to keep your clocks in sync.

One way Authentication

Authentication of one-way

Electronic mail (e-mail) is a common application for which encryption is growing. The very essence and key advantage of e-mail is that the sender and recipient do not have to be online simultaneously. The e-mail message is sent instead to the electronic mailbox of the recipient, where it is tamped up before it can be read by the recipient.

In order to process the message using the store and forward e-mail protocol, such as the Simple Mail Transfer Protocol (SMTP) or x.400, you must be clear of the 'endbox' or header of the e-mail message. But the mail handling protocol is also preferable, since this requires trust in the mail handling process, and does not require a plaintext version of the message access. The e-mail message should also be encrypted to avoid the decryption key from being used by the mail-handling system.

The authentication's second prerequisite. The receiver generally requires some trust that the message is from the assumed sender.



Remote Authentication using Symmetric Encryption

Mutual Authentication

As mentioned in Chapter 14, symmetrically-coded keys can be used as a two-level hierarchy to provide confidentiality in a distributed environment. In general, the technique requires the use of a trustworthy KDC. The KDC shares a hidden key, called a master key, with each party on the Network. The KDC shall create keys for a short period of time for a

link between the two parties called session keys and distribute them using the master keys to guard the distribution. This is a rather popular approach. For example, in section 15.3 we discuss the Kerberos method. This subsection explores how the Kerberos processes are interpreted.

Figure below depicts the proposal for the secret key distribution using a KDC initially put forward by Needham and Schroeder [NEED78] and that includes authentication features as mentioned in Chapter 14. The protocol can be summarised accordingly1.

1. A S KDC: $ID_A \} ID_B \} N_1$
2. KDC S A: E(K_a,[$K_s \} ID_B \} N_1 \}$ E(K_b,[$K_s \} ID_A$])])
3. A S B: E(K_b, [$K_s \} ID_A$])
4. B S A: E(K_s, N_2)
5. A S B: E(K_s, f(N_2))

Secret keys K_a and K_b are shared between A and the KDC and B and the KDC, respectively. The purpose of the protocol is to distribute securely a session key K_s to A and B. A securely acquires a new session key in step 2. The message in step 3 can be decrypted, and hence understood, only by B. Step 4 reflects B's knowl- edge of K_s, and step 5 assures B of A's knowledge of K_s and assures B that this is a fresh message because of the use of the nonce N_2. Recall from our discussion in Chapter 14 that the purpose of steps 4 and 5 is to prevent a certain type of replay at- tack. In particular, if an opponent is able to capture the message in step 3 and replay it, this might in some fashion disrupt operations at B.

The protocol is still vulnerable to a form of playback, despite a handshake in steps 4 and 5. Suppose an adversary, X, might compromise an old session key. This is definitely much more unlikely than a clear observation and registration by an opponent of step 3. It's a possible safety threat, however. By simply replaying step 3, X will impersonate A and trick



B to use the old key. B would not be able to tell that it is a replay unless B permanently recalls all preceding main session keys used with A. If X can intercept the handshake message in step 4, A can be indicated in step 5. From then on, X will send fake messages with the session key to B that seem to be B from A.

Denning [DENN81, DENN82] proposes to overcome this weakness by a modification to the Needham/Schroeder protocol that includes the addition of a timestamp to steps 2 and 3. Her proposal assumes that the master keys, K_a and K_b, are secure, and it consists of the following steps.

1. A S KDC: $ID_A \} ID_B$
2. KDC S A: E(K_a, [$K_s \} ID_B \} T \}$ E(K_b, [$K_s \} ID_A \} T$])])
3. A S B: E(K_b, [$K_s \} ID_A \} T$])
4. B S A: E(K_s, N1)
5. A S B: E(K_s, f(N_1))

T is a timestamp that assures A and B that the session key has only just been generated. Thus, both A and B know that the key distribution is a fresh exchange. A and B can verify timeliness by checking that

0Clock – T06 $\Delta t_1 + \Delta t_2$

where Δt_1 is the estimated normal discrepancy between the KDC's clock and the local clock (at A or B) and Δt_2 is the expected network delay time. Each node can set its clock against some standard reference source. Because the timestamp T is encrypted using the secure master keys, an opponent, even with knowledge of an old session key, cannot succeed because a replay of step 3 will be detected by B as untimely.

A final point: Steps 4 and 5 were not included in the original presentation [1] but were added later [1]. These steps confirm the receipt of the session key at B.

Version 4 of Kerberos makes use of DES, in a rather elaborate protocol, to provide the authentication service. Viewing the protocol as a whole, it is difficult to see the need for the many elements contained therein. Therefore, we adopt a strategy used by Bill Bryant of Project Athena [1] and build up to the full protocol by look- ing first at several hypothetical dialogues. Each successive dialogue adds additional complexity to counter security vulnerabilities revealed in the preceding dialogue.

After examining the protocol, we look at some other aspects of version 4.

A Simple AuthenticAtion DiAlogue In an unprotected network environment, any client can apply to any server for service. The obvious security risk is that of im- personation. An opponent can pretend to be another client and obtain unauthor- ized privileges on server machines. To counter this threat, servers must be able to confirm the identities of clients who request service. Each server can be required to undertake this task for each client/server



interaction, but in an open environment, this places a substantial burden on each server.

An alternative is to use an authentication server (AS) that knows the pass- words of all users and stores these in a centralized database. In addition, the AS shares a unique secret key with each server. These keys have been distributed physi- cally or in some other secure manner. Consider the following hypothetical dialogue:

(1) C S AS: $ID_C \} P_C \} ID_V$

(2) AS S C: $Ticket$

(3) C S V: $ID_C \} Ticket\ Ticket = E(K_v, [ID_C \} AD_C \} ID_V])$

where

C = client

AS = authentication server V = server

ID_C = identifier of user on C

ID_V = identifier of V

P_C = password of user on C

AD_C = network address of C

K_v = secret encryption key shared by AS and V

In this scenario, the user logs on to a workstation and requests access to server V. The client module C in the user's workstation requests the user's password and then sends a message to the AS that includes the user's ID, the server's ID, and the user's password. The AS checks its database to see if the user has supplied the proper password for this user ID and whether this user is permitted access to server V. If both tests are passed, the AS accepts the user as authentic and must now convince the server that this user is authentic. To do so, the AS creates a **ticket** that con- tains the user's ID and network address and the server's ID. This ticket is encrypted using the secret key shared by the AS and this server. This ticket is then sent back to C. Because the ticket is encrypted, it cannot be altered by C or by an opponent.

With this ticket, C can now apply to V for service. C sends a message to V con- taining C's ID and the ticket. V decrypts the ticket and verifies that the user ID in the ticket is the same as the unencrypted user ID in the message. If these two match, the server considers the user authenticated and grants the requested service.

Each of the ingredients of message (3) is significant. The ticket is encrypted to prevent alteration or forgery. The server's ID (ID_V) is included in the ticket so that the server can verify that it has decrypted the ticket properly. ID_C is included in the ticket to indicate that this ticket has been issued on behalf of C. Finally, AD_C serves to counter the following threat. An opponent could capture the ticket transmitted in message (2), then use the name ID_C and transmit a message of form (3) from another workstation. The server would receive a valid



ticket that matches the user ID and grant access to the user on that other workstation. To prevent this attack, the AS includes in the ticket the network address from which the original request came. Now the ticket is valid only if it is transmitted from the same workstation that initially requested the ticket.

A more Secure Authentication Dialogue Although the foregoing scenario solves some of the problems of authentication in an open network environment, problems remain. Two in particular stand out. First, we would like to minimize the number of times that a user has to enter a password. Suppose each ticket can be used only once. If user C logs on to a workstation in the morning and wishes to check his or her mail at a mail server, C must supply a password to get a ticket for the mail server. If C wishes to check the mail several times during the day, each attempt requires reentering the password. We can improve matters by saying that tickets are reusable. For a single logon session, the workstation can store the mail server ticket after it is received and use it on behalf of the user for multiple accesses to the mail server.

However, under this scheme, it remains the case that a user would need a new ticket for every different service. If a user wished to access a print server, a mail server, a file server, and so on, the first instance of each access would require a new ticket and hence require the user to enter the password.

The second problem is that the earlier scenario involved a plaintext transmis- sion of the password [message (1)]. An eavesdropper could capture the password and use any service accessible to the victim.

To solve these additional problems, we introduce a scheme for avoiding plain- text passwords and a new server, known as the **ticket-granting server** (TGS). The new (but still hypothetical) scenario is as follows.

Once per user logon session:

(1) C S AS: $ID_C \} ID_{tgs}$

(2) AS S C: E(K_c, $Ticket_{tgs}$)

Once per type of service:

(3) C S TGS: $ID_C \} ID_V \} Ticket_{tgs}$

(4) TGS S C: $Ticket_v$

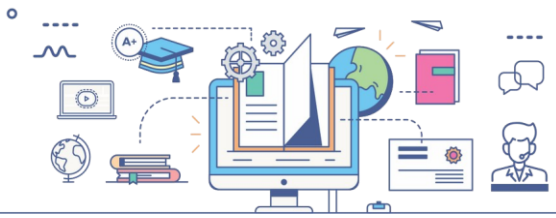
Once per service session:

(5) C S V: $ID_C \} Ticket_v$

$Ticket_{tgs}$ = E(K_{tgs}, [$ID_C \} AD_C \} ID_{tgs} \} TS_1 \} Lifetime_1$]) $Ticket_v$ = E(K_v, [$ID_C \} AD_C \} ID_v \} TS_2 \} Lifetime_2$])

The new service, TGS, issues tickets to users who have been authenticated to AS. Thus, the user first requests a ticket-granting ticket ($Ticket_{tgs}$) from the AS. The client module in the user workstation saves this ticket. Each time the user requires access to a new service, the client applies to the TGS, using the ticket to authenti- cate itself. The TGS then grants a ticket for the particular service. The client saves each service-granting ticket and uses it to authenticate its user to a server each time a particular service is requested. Let us look at the details of this scheme:

1. The client requests a ticket-granting ticket on behalf of the user by sending its



user's ID to the AS, together with the TGS ID, indicating a request to use the TGS service.

2. The AS responds with a ticket that is encrypted with a key that is derived from the user's password (K_c), which is already stored at the AS. When this response arrives at the client, the client prompts the user for his or her pass- word, generates the key, and attempts to decrypt the incoming message. If the correct password is supplied, the ticket is successfully recovered.

Because only the correct user should know the password, only the correct user can recover the ticket. Thus, we have used the password to obtain credentials from Kerberos without having to transmit the password in plaintext. The ticket itself consists of the ID and network address of the user, and the ID of the TGS. This corresponds to the first scenario. The idea is that the client can use this ticket to request multiple service-granting tickets. So the ticket-granting ticket is to be reus- able. However, we do not wish an opponent to be able to capture the ticket and use it. Consider the following scenario: An opponent captures the login ticket and waits until the user has logged off his or her workstation. Then the opponent either gains access to that workstation or configures his workstation with the same net- work address as that of the victim. The opponent would be able to reuse the ticket

To spoof the TGS. To counter this, the ticket includes a timestamp, indicating the date and time at which the ticket was issued, and a lifetime, indicating the length of time for which the ticket is valid (e.g., eight hours). Thus, the client now has a reusable ticket and need not bother the user for a password for each new service request. Finally, note that the ticket-granting ticket is encrypted with a secret key known only to the AS and the TGS. This prevents alteration of the ticket. The ticket is reencrypted with a key based on the user's password. This assures that the ticket can be recovered only by the correct user, providing the authentication.
Now that the client has a ticket-granting ticket, access to any server can be obtained with steps 3 and 4.

3.The client requests a service-granting ticket on behalf of the user. For this pur- pose, the client transmits a message to the TGS containing the user's ID, the ID of the desired service, and the ticket-granting ticket.

4.The TGS decrypts the incoming ticket using a key shared only by the AS and the TGS (K_{tgs}) and verifies the success of the decryption by the presence of its ID. It checks to make sure that the lifetime has not expired. Then it compares the user ID and network address with the incoming information to authenti- cate the user. If the user is permitted access to the server V, the TGS issues a ticket to grant access to the requested service.

The service-granting ticket has the same structure as the ticket-granting ticket. Indeed, because the TGS is a server, we would expect that the same elements are needed to authenticate a client to the TGS and to authenticate a client to an appli- cation server. Again, the ticket contains a timestamp and lifetime. If the user wants access to the same service at a later time, the client can simply use the previously acquired service-granting ticket and need not bother the user for a password. Note that the ticket is encrypted with a secret key (K_v) known only to the TGS and the server, preventing alteration.

Finally, with a particular service-granting ticket, the client can gain access to the corresponding service with step 5.



5.The client requests access to a service on behalf of the user. For this purpose, the client transmits a message to the server containing the user's ID and the service- granting ticket. The server authenticates by using the contents of the ticket.

This new scenario satisfies the two requirements of only one password query per user session and protection of the user password.

Version 4 Authentication Dialogue Although the foregoing scenario en- hances security compared to the first attempt, two additional problems remain.

The heart of the first problem is the lifetime associated with the ticket-granting ticket. If this lifetime is very short (e.g., minutes), then the user will be repeatedly asked for a password. If the lifetime is long (e.g., hours), then an opponent has a greater opportunity for replay. An opponent could eavesdrop on the network and capture a copy of the ticket-granting ticket and then wait for the legitimate user to log out. Then the opponent could forge the legitimate user's network address and send the message of step (3) to the TGS. This would give the opponent

unlimited access to the resources and files available to authenticate user.

Similarly, if an opponent captures a service-granting ticket and uses it before it expires, the opponent has access to the corresponding service.
Thus, we arrive at an additional requirement. A network service (the TGS or an application service) must be able to prove that the person using a ticket is the same person to whom that ticket was issued.
The second problem is that there may be a requirement for servers to authenticate themselves to users. Without such authentication, an opponent could sabotage the configuration so that messages to a server were directed to another location. The false server would then be in a position to act as a real server and capture any information from the user and deny the true service to the user.
We examine these problems in turn and refer to Table 8.1, which shows the actual Kerberos protocol. Figure 8.1 provides a simplified overview.

First, consider the problem of captured ticket-granting tickets and the need to determine that the ticket presenter is the same as the client for whom the ticket was issued. The threat is that an opponent will steal the ticket and use it before it expires. To get around this problem, let us have the AS provide both the client and the TGS with a secret piece of information in a secure manner. Then the client can prove its identity to the TGS by revealing the secret information—again in a secure manner. An efficient way of accomplishing this is to use an encryption key as the secure information; this is referred to as a session key in Kerberos.

Table 8.1a shows the technique for distributing the session key. As before, the client sends a message to the AS requesting access to the TGS. The AS responds with a message, encrypted with a key derived from the user's password (K_c), that contains the ticket. The encrypted message also contains a copy of the session key, $K_{c,tgs}$, where the subscripts indicate that this is a session key for C and TGS. Because this session key is inside the message encrypted with K_c, only the user's client can read it. The same session key is included in the ticket, which can be read only by the TGS. Thus, the session key has been securely delivered

to both C and the TGS.

Table 8.1 Summary of Kerberos Version 4 Message Exchanges

(1) C S **AS** $ID_c \| ID_{tgs} \| TS_1$
(2) **AS** S C $E(K_c, [K_{c,tgs} \| ID_{tgs} \| TS_2 \| Lifetime_2 \| Ticket_{tgs}])$
$Ticket_{tgs} = E(K_{tgs}, [K_{c,tgs} \| ID_C \| AD_C \| ID_{tgs} \| TS_2 \| Lifetime_2])$

(a) Authentication Service Exchange to obtain ticket-granting ticket

(3) **C S TGS** $ID_v \| Ticket_{tgs} \| Authenticator_c$
(4) **TGS S C** $E(K_{c,tgs}, [K_{c,v} \| ID_v \| TS_4 \| Ticket_v])$
$Ticket_{tgs} = E(K_{tgs}, [K_{c,tgs} \| ID_C \| AD_C \| ID_{tgs} \| TS_2 \| Lifetime_2])$ $Ticket_v$
$= E(K_v, [K_{c,v} \| ID_C \| AD_C \| ID_v \| TS_4 \| Lifetime_4])$ $Authenticator_c =$
$E(K_{c,tgs}, [ID_C \| AD_C \| TS_3])$

(b) Ticket-Granting Service Exchange to obtain service-granting ticket

(5) **C S V** $Ticket_v \| Authenticator_c$
(6) **V S C** $E(K_{c,v}, [TS_5 + 1])$ (for mutual authentication)
$Ticket_v = E(K_v, [K_{c,v} \| ID_C \| AD_C \| ID_v \| TS_4 \| Lifetime_4])$
$Authenticator_c = E(K_{c,v}, [ID_C \| AD_C \| TS_5])$

(c) Client/Server Authentication Exchange to obtain service

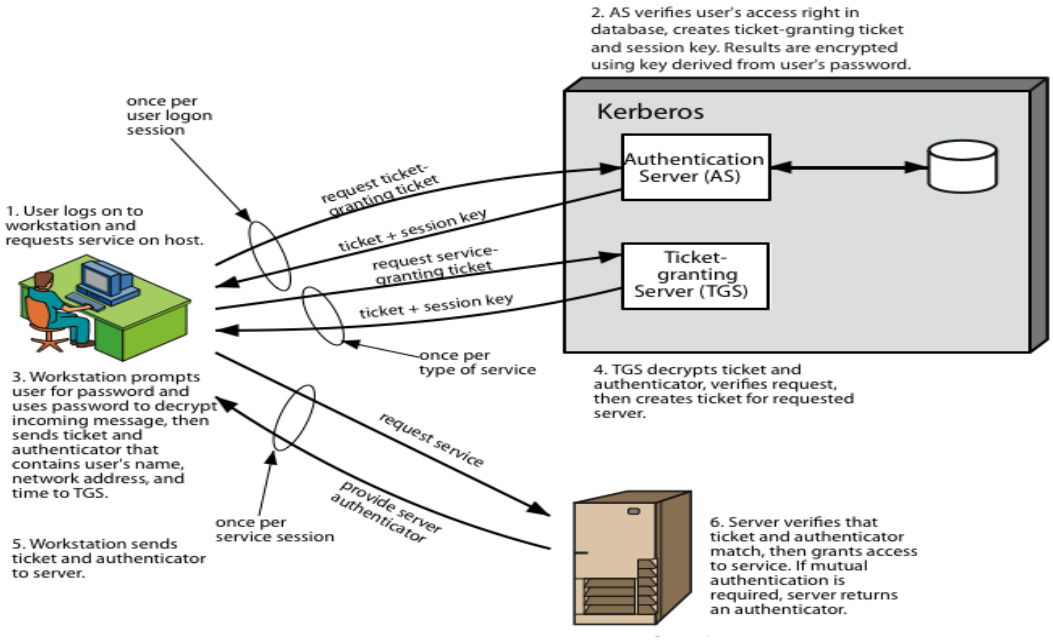


Fig-8.2 Kerberos Protocol(Image Source: Ref[1])

KerberOs Realm

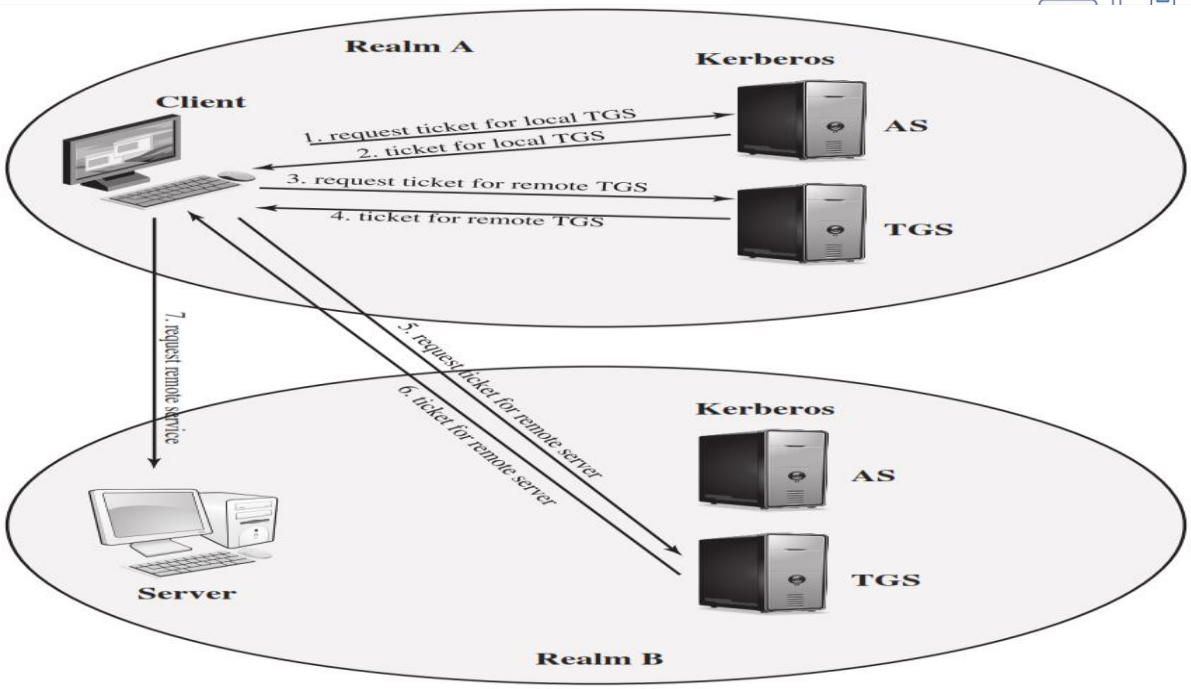
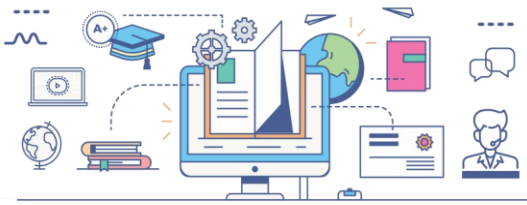


Fig-8.4 Kerberos Realm(Image Source: Ref[1])

KerberOS V5

- **It was developed in mid 1990's**
- **This version is specified as Internet standard RFC 1510**
- **This Version provides improvements over v4**
- **IT addresses environmental shortcomings**
- **encryption algo, network protocol, byte order, ticket lifetime, authentication forwarding and technical deficiencies are as shown below.**



(1) C → AS Options || ID_c || $Realm_c$ || ID_{tgs} || $Times$ || $Nonce_1$

(2) AS → C $Realm_c$ || ID_C || $Ticket_{tgs}$ || $E(K_c, [K_{c,tgs} || Times || Nonce_1 || Realm_{tgs} || ID_{tgs}])$

$Ticket_{tgs} = E(K_{tgs}, [Flags || K_{c,tgs} || Realm_c || ID_C || AD_C || Times])$

(a) Authentication Service Exchange to obtain ticket-granting ticket

(3) C → TGS Options || ID_v || $Times$ || || $Nonce_2$ || $Ticket_{tgs}$ || $Authenticator_c$

(4) TGS → C $Realm_c$ || ID_C || $Ticket_v$ || $E(K_{c,tgs}, [K_{c,v} || Times || Nonce_2 || Realm_v || ID_v])$

$Ticket_{tgs} = E(K_{tgs}, [Flags || K_{c,tgs} || Realm_c || ID_C || AD_C || Times])$

$Ticket_v = E(K_v, [Flags || K_{c,v} || Realm_c || ID_C || AD_C || Times])$

$Authenticator_c = E(K_{c,tgs}, [ID_C || Realm_c || TS_1])$

(b) Ticket-Granting Service Exchange to obtain service-granting ticket

(5) C → V Options || $Ticket_v$ || $Authenticator_c$

(6) V → C $E_{Kc,v}$ [TS_2 || Subkey || Seq#]

$Ticket_v = E(K_v, [Flags || K_{c,v} || Realm_c || ID_C || AD_C || Times])$

$Authenticator_c = E(K_{c,v}, [ID_C || Realm_c || TS_2 || Subkey || Seq\#])$

(c) Client/Server Authentication Exchange to obtain service

Fig-8.6 Kerberos v5(Image Source: Ref[1])

- **it uses double encryption, non-std mode of use, session keys, password attacks are possible in this**

Remote Authentication Using Assymetric Encryption

Mutual Authentication Protocol

1. A → AS: $ID_A || ID_B$
2. AS → A: $E(PR_{as}, [ID_A || PU_a || T]) || E(PR_{as}, [ID_B || PU_b || T])$
3. A → B: $E(PR_{as}, [ID_A || PU_a || T]) || E(PR_{as}, [ID_B || PU_b || T]) || E(PU_b, E(PR_a, [K_s || T]))$

In this case, the central system is referred to as an authentication server (AS), because it is not actually responsible for secret-key distribution. Rather, the AS pro- vides public-key certificates. The session key is chosen and encrypted by A; hence, there is no risk of exposure by the AS. The timestamps protect against replays of compromised keys.



This protocol is compact but, as before, requires the synchronization of clocks. Another approach, proposed by Woo and Lam [1], makes use of nonces. The protocol consists of the following steps.

Asymmetric Message Exchanges

1. A → KDC: $ID_A \| ID_B$
2. KDC → A: $E(PR_{auth}, [ID_B \| PU_b])$
3. A → B: $E(PU_b, [N_a \| ID_A])$
4. B → KDC: $ID_A \| ID_B \| E(PU_{auth}, N_a)$
5. KDC → B: $E(PR_{auth}, [ID_A \| PU_a]) \| E(PU_b, E(PR_{auth}, [N_a \| K_s \| ID_B]))$
6. B → A: $E(PU_a, [E(PR_{auth}, [(N_a \| K_s \| ID_B)]) \| N_b])$
7. A → B: $E(K_s, N_b)$

A informs the KDC of its intention to establish a secure connection with B. The KDC returns to A a copy of B's public-key certificate (step 2). Using B's public key, A informs B of its desire to communicate and sends a nonce N_a (step 3). In step 4, B asks the KDC for A's public-key certificate and requests a session key; B includes A's nonce so that the KDC can stamp the session key with that nonce.

The nonce is protected using the KDC's public key. In step 5, the KDC returns to B a copy of A's public-key certificate, plus the information $\{N_a, K_s, ID_B\}$. This infor- mation basically says that K_s is a secret key generated by the KDC on behalf of B and tied to N_a; the binding of K_s and N_a will assure A that K_s is fresh. This triple is encrypted using the KDC's private key to allow B to verify that the triple is in fact from the KDC. It is also encrypted using B's public key so that no other entity may use the triple in an attempt to establish a fraudulent connection with A. In step 6, the triple $\{N_a, K_s, ID_B\}$, still encrypted with the KDC's private key, is relayed to A, together with a nonce N_b generated by B.

All the foregoing are encrypted using A's public key. A retrieves the session key K_s, uses it to encrypt N_b, and returns it to B. This last message assures B of A's knowledge of the session key.